Dictionary Server Report

Han Perry 693878

Introduction

This report describes the design and implementation of `DictionaryServer` and `DictionaryClient`. The server is a multi-threaded application that provides a dictionary service to clients, allowing a client to query the meaning(s) of a word, add a new word, remove an existing word, add a new meaning to a word and additional features such as editing and removing a selected meaning from a word. The server is implemented in Java and uses sockets for communication with clients. The server is capable of handling multiple clients concurrently through multithreading. The server uses a MySQL database to store the dictionary data. On the client side, there is a user-friendly graphical user interface.

Sockets and Networking

Internet Protocol

The server and client communicate using the Transmission Control Protocol (TCP), chosen for its reliability and guarantee of lossless message arrival [3].

Server

The server initialises a listening socket set to the port number specified by the user, and listens for incoming client connections. When a client connect, a `WorkerRunnable` object is created with the accepted client socket and added to the threadpool queue to be executed. Once processed, the worker thread sends a response back to the client and closes the connection.

Client

The `ClientConnect` class handles the client-side socket connection. For each action (e.g. a word query), the client will establish a new connection with the server. Because the actions performed are fairly simple, this short-lived connection is an acceptable design choice and decreases the risk of an accidental disconnection. The overhead of establishing a new connection for each action should be reasonable as the client is not expected to perform a large number of actions in quick succession. When the client sends a request to the server it then waits for a response, which acts as a way to reduce the risk of spam actions from a single unmodified client.

Since both client and server use the `writeUTF` and `readUTF` methods, the dictionary is compatible with unicode characters, including emoji.

Concurrency and Multithreading

Architecture

The server utilises the Java Executor Framework [7] to manage a thread pool of worker threads, allowing multiple clients to connect concurrently. The server uses `ThreadPoolExecutor` to create a customised thread pool architecture, based on a cached thread pool [6] but with additional constraints of a minimum and maximum numbers of threads. When required, new threads are created up to the maximum number of threads, previously constructed threads are reused when available, and idle threads are terminated after a specified time (set to 60 seconds). A small core pool size of five is also set as a way to maintain some threads in the pool even when idle. This architecture allows the server to handle a usual load of clients concurrently with reduced overhead for thread creation, while also being able to scale up to meet increased demand. A maximum thread count of 20 is set to prevent resource exhaustion in the case of a large number of clients connecting simultaneously. Although this maximum protects against resource exhaustion, it limits the server's ability to handle a large number of clients concurrently if the actions requested are long-running. However, this is an acceptable trade-off as the server is designed for simple dictionary tasks that generally do not take a long time to process.

Message Protocols

The server and client communicate using the `Message` class, which is a data structure that contains the information for the selected action. The `command` is a string that specifies the action that the client is requesting or the action request that the server is responding to. The `word` is the string word that the action is being carried out on, and `meaning` and `oldMeaning` are both single meanings referring to the word (the former being used in a message where it is requesting for an existing meaning to be edited). The `meanings` arraylist is used to send the multiple meaning of a word. For error handling there is an `error` string that is used to send details of the issue back to the client if an error occurs, `success` for a boolean indicator of success or failure, and `valid` for message format validation. The `Message` class is shown in Figure 1.

The message is serialised into a string using the `toString` method of the `Message` class, which uses the `ObjectMapper` from the jackson library [4] to convert the message object into a JSON formatted string that can be sent between server and client. The JSON string is then sent and received using the `writeUTF` and `readUTF` methods of the `DataOutputStream` and `DataInputStream` classes, respectively.

As the message format is checked for validity, the server is protected against malformed messages that could cause errors, such as a message with a null value for a required field. The server will respond by sending an error message back to the client, which will then display the error message to the user. This design choice was made to ensure that the server is robust and can handle unexpected input from the client.

Dictionary Implementation

The dictionary is stored using a mySQL database for dictionary storage and Java JDBC [8] for interacting with this database [2] within the `MySqlHandler` class. A mySQL database interfaced

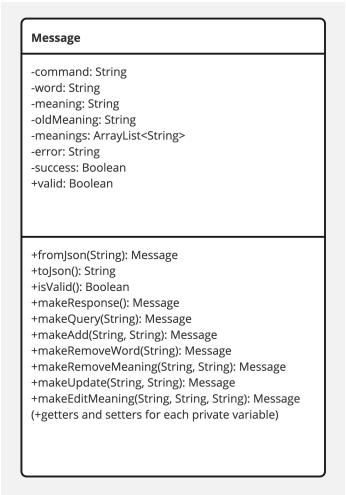


Figure 1: Message class

by JDBC is more suitable than a file or in-memory object as it allows for scalability of the number of words and meanings stored while maintaining a reasonably fast and efficient lookup. By using a separate database connection for each prepared statement, the server can handle multiple clients concurrently without the risk of data corruption in the database. The mySQL handler class uses prepared statements to prevent SQL injection attacks.

Database Schema

The database schema consists of a single table, `dictionary`, with columns `id`, `word` and `meaning`. The `id` column is the primary key, so a word can have multiple entries corresponding to different meanings. The schema is shown in Figure 2. While this is a simple schema, it is sufficient for the requirements of the dictionary server. Using a database also gives flexibility to expand the schema to include additional features in the future, such as user accounts and permissions.

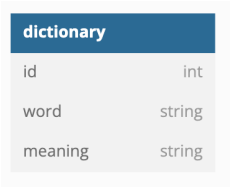
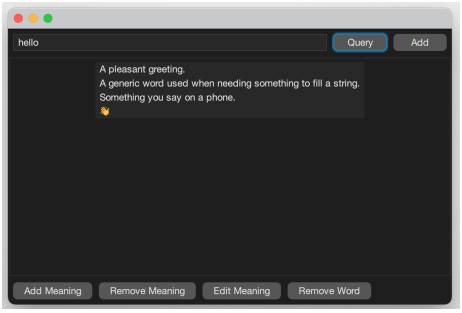


Figure 2: Database schema

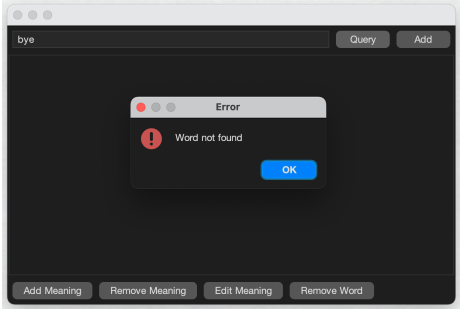
Graphical User Interface

Client

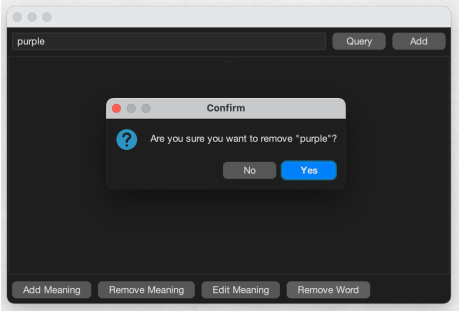
For ease of usability, the dictionary client has a graphical user interface (GUI) that allows the user to type a word into the text field and select the desired action. The GUI is implemented using Java Swing [9] and the FlatLaf look and feel to give the GUI a sleek darkmode appearance [5]. A collection of screenshots displaying some of the various features of the client GUI is shown in Figure 3.



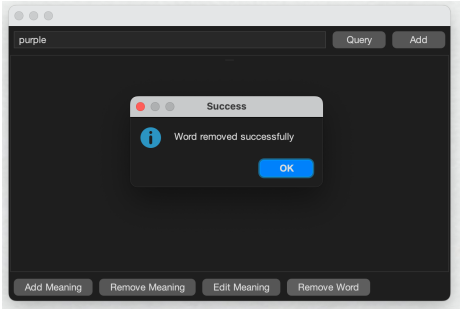
(a) Displaying meanings of a word



(b) Error message



(c) Confirmation

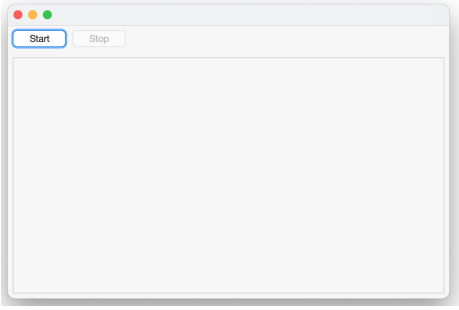


(d) Information

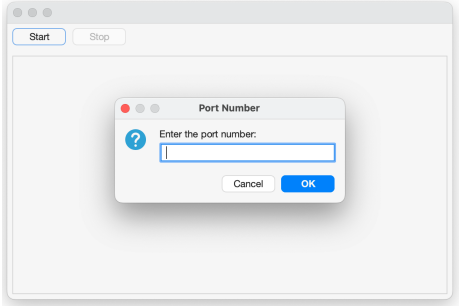
Figure 3: Client GUI

Server

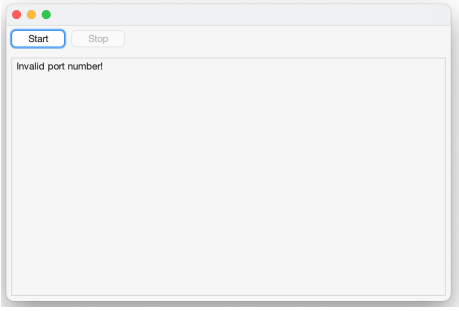
The server also has a simple GUI that allows the user to start and stop the server and view a log of events and server-based errors that arise. The server GUI is also implemented using Java Swing [9] and the FlatLaf look and feel to give a simple lightmode appearance [5]. The server GUI is shown in Figure 4.



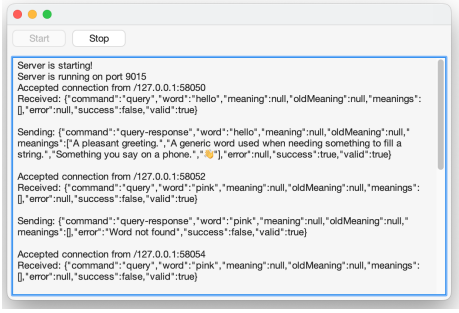
(a) Initial view on launch



(b) Asking for port number



(c) When an issue occurs



(d) While running

Figure 4: Server GUI

Error Handling

In order for the dictionary to be robust and reliable, error handling is implemented at both the client and server side.

Client-side

If an issue can be detected on the client side, the client will avoid sending the malformed message to the server and instead will display an error message to the user. This includes issues such as pressing the query or add button without typing a word into the text field or leaving the text field blank when prompted for a meaning to add when attempting to add a new word or add a new meaning to a word.

If the server detects an issue or an error is raised during the processing of the client's request message, the server sends an error message back to the client and the client displays the error message to the user with an error pop-up alert, allowing the user to understand what went wrong. This includes issues such as a word not being found in the database, a word already existing in the database when trying to add a new word, a meaning being added to a word that already exists, or a meaning not being found when trying to edit or remove a meaning from a word. Since these issues are not known to the client before sending the action request message, the server must handle these errors and it cannot be prevented solely on the client side.

If the server cannot be reached by the client, the client will display an error message to the user, informing them that it was unable to fulfil their requested action and to check the connection to the server.

When launched from the command line, the client will display an error message if the server address or port number is not provided as command line arguments.

Server-side

The server performs checks to ensure the required fields of the message are provided and that the action being requested by the client is valid and appropriate.

The server also handles errors that arise when interacting with the database, such as not being able to establish a connection, and sends an informative error message back to the client if an error occurs.

Logging

The server logs interactions using log4j2 [1] to help with debugging and monitoring. This provides a log of each requested command, the word specified and the ip address of the client, that persists after the server is shut down. This allows for monitoring of misuse of the dictionary, such as malicious editing of the dictionary or spamming the server with messages. Log4j2 is designed to be fast and efficient, with lock-free asynchronous logging to prevent the logging from slowing down the server's performance and allow it to be used in contexts requiring concurrency.

An additional log is visible in the server GUI, which displays the log of events and server-based errors that arise. This provides a user-friendly way to monitor the server's status and view any errors that occur.

Excellence

There are several aspects of this project's implementation which go beyond what was required in the specification, including:

- **Dependency management:** Using Maven for dependency management with a multi-module project structure [10]. This allowed for simplified dependency management and project structure.
- **Error handling:** The server and client both have extensive error handling to ensure the dictionary is robust and reliable. The user is informed of any errors that occur and given details that help them resolve the issue.
- **Logging:** The server logs interactions using log4j2 to help with debugging and monitoring.
- **Database:** Using a MySQL database for dictionary storage and JDBC for interacting with the database. This allows for scalability of the size of the dictionary while maintaining a reasonably fast and efficient lookup.
- **Concurrency:** The server uses a customised thread pool architecture to handle multiple clients concurrently, allowing the server to scale up to meet increased demand while balancing scalability with resource capacity and limitations.

Creativity

This project implements additional aspects that demonstrate creativity, including:

- **Remove meaning:** This additional command gives the ability to remove a selected meaning from a word, without needing to remove the entire word. This feature is not present in the specification but adds additional functionality to the dictionary. The user interface allows for clicking on the meaning to select it and a convenient 'Remove Meaning' is provided.
- **Edit meaning:** This additional command allows the user to edit an existing meaning of a word. This feature is also not present in the specification but adds additional functionality and usability to the dictionary. When the user selects a meaning and clicks 'Edit Meaning' an input box pops up with the existing meaning pre-filled, allowing the user to edit the meaning and submit the changes to the dictionary.
- **Server GUI:** The server has a simple GUI that allows the user to start and stop the server and view a log of events and server-based errors that arise. This provides a user-friendly way to interact with the server and monitor its status.

References

[1] Apache. *Apache Log4j*. 2024. URL: `https://logging.apache.org/log4j/2.x/` (visited on 03/30/2024).

[2] Vogella Blog. *MySQL and Java JDBC - Tutorial*. 2024. URL: `https://www.vogella.com/tutorials/MySQLJava/article.html` (visited on 03/30/2024).

[3] W. Eddy. *RFC 9293: Transmission Control Protocol (TCP)*. USA, 2022.

[4] FasterXML. *Jackson project*. 2024. URL: https://github.com/FasterXML/jackson (visited on 04/07/2024).

[5] Formdev. *Flatlaf Themes*. 2024. URL: https://www.formdev.com/flatlaf/themes/ (visited on 04/07/2024).

[6] GeeksforGeeks. *Thread Pools in Java*. 2024. URL: https://www.geeksforgeeks.org/thread-pools-java/ (visited on 03/30/2024).

[7] Oracle. *Concurrent Package Summary*. 2024. URL: https://docs.oracle.com/javase/21/docs/api/java/util/concurrent/package-summary.html (visited on 03/30/2024).

[8] Oracle. *JDBC Developer's Guide and Reference*. 2022. URL: https://docs.oracle.com/en/database/oracle/oracle-database/21/jjdbc/ (visited on 03/30/2024).

[9] Oracle. *Swing Java Package Documentation*. 2024. URL: https://docs.oracle.com/en/java/javase/21/docs/api/java.desktop/javax/swing/package-summary.html (visited on 04/06/2024).

[10] Apache Maven Project. *POM reference*. 2024. URL: https://maven.apache.org/pom.html (visited on 03/30/2024).